CS 621

Scalable AI Services

w/ Agentic AI, MLOps & LLMOps

Quiz 1 — Complete Study Notes

Prompt Engineering · RAG · Data Prep for RAG · LangChain Agents

Table of Contents

Module 1: Prompt Engineering

Techniques, Tips & Best Practices

1.1 What is a Prompt?

Prompt: An input or query given to a large language model (LLM) to elicit a specific response or output.

Prompt Engineering: The practice of designing and refining prompts to optimize the responses generated by AI models.

1.2 Components of a Good Prompt

A well-structured prompt typically has 4 components:

Component	Definition	Example
Instruction	A clear directive telling the model what to do	Summarize the following article and list top 3 points...
Context	Background/additional info to understand the task	The article is about climate change and polar bears
Input / Question	The specific query or data for the model to process	"How is climate change affecting polar bear habitats?"
Output Type / Format	Desired structure or style of the response	In markdown format as a numbered list

1.3 Prompt Engineering Techniques

Zero-Shot Prompting

A prompt that asks the model to perform a task WITHOUT providing any examples.

- The model relies entirely on its pre-trained knowledge.
- Best for: simple, well-understood tasks.

Example: "Classify the following review into neutral, negative or positive:
Review: 'I absolutely loved this movie!' Sentiment: ..."

Few-Shot Prompting

A prompt that provides a FEW input-output examples to guide the model to generate the desired output.

- Helps model understand the expected pattern/format.
- More reliable than zero-shot for complex tasks.

Example: Give 2–3 labeled sentiment examples BEFORE asking the model to classify a new review.

Chain-of-Thought (CoT) Prompting

Enhances reasoning by asking the model to articulate its thought process step-by-step, similar to human reasoning.

- Triggered by phrases like: *"Let's think step-by-step"*
- Helps avoid wrong answers on math/logic problems.
- Research note: findings are mixed — some models don't follow CoT faithfully.

Without CoT	With CoT
"The bakery had 23 bananas. Used 20, bought 6 more. How many?" → The answer is 27. ■	"...Let's think step-by-step." → 23-20=3, 3+6=9. The answer is 9. ■

Prompt Chaining

Multiple tasks are linked together — the output of one prompt serves as the input for the next.

- Breaks complex tasks into manageable subtasks.
- Output is passed downstream automatically.

  Example use case: Step 1: Extract relevant quotes from a document → Step 2: Use those quotes to answer a question.

1.4 Prompt Engineering Tips & Best Practices

Prompts are model-specific

- Different models may require different prompts — always test.
- Iterative development is key: refine prompts through testing.
- Adjust temperature: higher = more creative, lower = more focused/deterministic.
- Be aware of bias and hallucination risks.

Format your prompts

- Use delimiters to separate instruction from context: ### or ``` or {} or ---
- Ask model to return structured output: JSON, HTML, Markdown, table, etc.
- Provide a correct example: 'Output should look like {"Title": "In and Out"}'

Guide the model for better responses

- "Do not make things up if you do not know. Say: I do not have that information."
- "Do not make assumptions based on nationalities."
- "Do not ask the user to provide their SSNs."
- "Do this step-by-step. Step 1: Summarize into 100 words. Step 2: Translate to French..."

1.5 Benefits & Limitations of Prompt Engineering

Benefits	Limitations
Simple and efficient — fast to implement	Output quality depends on the model used
Predictable results — meets predefined accuracy standards	Limited by the pre-trained model's internal knowledge

Tailored outputs — customize AI responses to specific needs/styles	For external/proprietary knowledge, RAG is needed

■■ **Important:** Prompt engineering is LIMITED by the model's training data. If you need the model to answer using your own documents or up-to-date information, you MUST use RAG.

Module 2: Introduction to RAG

Retrieval Augmented Generation

2.1 How Do LLMs Learn Knowledge?

Method	Description	Requirements	Complexity
Pre-Training	Train LLM from scratch on massive datasets	Billions to trillions of tokens	Very High
Fine-Tuning	Adapt a pre-trained LLM to specific domains/tasks	Thousands of domain-specific examples	Medium
Passing Contextual Info (RAG)	Combine LLM with external knowledge retrieval	External knowledge base	Low ← Course Focus

■ **Tip:** RAG is preferred in this course because it has the LEAST complexity and compute-intensiveness while still providing accurate, up-to-date responses.

2.2 What is RAG?

RAG (Retrieval Augmented Generation) is a pattern (Lewis et al. 2020) that improves the efficacy of LLM applications by leveraging custom/external data.

It works by retrieving data/documents relevant to a question and providing them as context to augment the prompt to an LLM to improve generation.

The main problem RAG solves is the **knowledge gap** — when an LLM's training data does not include the information needed to answer a query accurately.

2.3 Passing Context: Benefits & Downsides

Passing context (like open notes in an exam) greatly improves factual recall. However, long contexts have downsides:

Downside	Explanation
Higher API costs	Costs are based on the number of input tokens
Longer inference times	More tokens = slower response generation
"Lost in the Middle" problem	LLMs tend to overlook documents placed in the middle of a long context (needle in haystack test)

2.4 Core Components of the RAG Workflow

Index & Embed	An embedding model creates vector representations of documents and user queries.

Vector Store	Specialized database to store unstructured data indexed by vectors (vector DB, library, or plugin).
Retrieval	Search stored vectors using similarity search to retrieve relevant chunks/documents.
Filtering & Reranking	Process of selecting or ranking retrieved documents before passing as context.
Prompt Augmentation	Inject retrieved data into the prompt to enhance context for the LLM.
Generation	The LLM uses the augmented prompt to generate a final response for the user.

2.5 RAG Architecture Flow

The RAG pipeline has two parallel paths that converge at the vector store:

→ **Document Embedding Path:** Source documents → Embedding Model → Vector representations stored in Vector Store

→ **User Query Path:** User question → Embedding Model → Query vector → Similarity Search against Vector Store → Retrieve top-k similar chunks

→ **Generation:** Retrieved chunks + original query → Augmented Prompt → LLM → Final Answer to user

2.6 Benefits of RAG Architecture

Benefit	Explanation
Up-to-date & Accurate Responses	LLM uses external data, not just static training data. Always current.
Reduces Hallucinations	Grounds responses in actual retrieved documents; outputs can cite original sources.
Domain-Specific Contextualization	Tailored to proprietary or domain-specific data, improving accuracy.
Efficiency & Cost-Effectiveness	Alternative to expensive fine-tuning; no up-front overhead; easy to update data.

2.7 Databricks Features for RAG

RAG Step	Databricks Feature	Details
Find Relevant Context	Mosaic AI Vector Search	Built into Databricks; integrated with governance; auto-syncs with Delta tables
Generate Response	Mosaic AI Model Serving	Foundation models (Llama-3), External models (GPT-4); Mosaic AI Playground for testing

Serve RAG Chain	Mosaic AI Model Serving	Unified deploy interface; low-latency, high-availability; scales up/down automatically
Document Ingestion	Delta Live Tables	Batch and streaming ingestion of source documents
Governance	Unity Catalog	Access controls, lineage, auditing, data sharing for RAG apps

Module 3: Preparing Data for RAG

Chunking, Embeddings & Data Governance

3.1 Why is Data Prep Important?

Poor data preparation leads to serious issues:

Issue	Impact
Poor quality model output	Inaccurate, incomplete, or biased data → misleading or incorrect responses
"Lost in the Middle"	Long contexts cause LLMs to overlook documents placed in the middle
Inefficient retrieval	Poorly prepared data decreases accuracy of retrieving relevant information
Exposing sensitive data	Poor data governance can expose private data during retrieval
Wrong embedding model	Incorrect model lowers quality of embeddings and retrieval accuracy

3.2 Data Prep Process Overview

The data preparation pipeline has 3 main stages:

Stage 1: Data Storage & Governance	Ingest from external sources → Store in Delta Tables/Volumes → Apply Unity Catalog governance
Stage 2: Data Extraction & Chunking	Parse and clean documents → Split into chunks (Fixed-size or Semantic) → Apply chunking strategy
Stage 3: Embedding	Pass chunks through embedding model → Generate vector representations → Store in Vector Store

3.3 Data Storage & Governance

Delta Lake: A unified data management layer providing data reliability and fast analytics. It is the optimized storage layer for Databricks.

Delta Lake features: Data Quality (consistency, reliability), Data Lineage (version tracking, governance), Featurization.

Unity Catalog (UC): Provides unified governance for RAG applications.

- Discovery — find data assets easily
- Access Controls — who can access what
- Lineage — track data origins
- Monitoring — observe data quality
- Auditing — log who accessed what
- Data Sharing — open sharing across teams

3.4 Chunking Strategies

Chunking = splitting documents into smaller pieces before embedding. The strategy MUST match your use case.

Strategy	Description	Best For	Tradeoff
Fixed-Size Chunking	Divide by a specific number of tokens	Simple documents, fast processing	May cut sentences mid-thought
Semantic Chunking	Split by sentence/paragraph/section using punctuation (., \n)	Preserving meaning and context	More complex to implement
Chunk Overlap	Consecutive chunks share some tokens to preserve context at boundaries	When context continuity matters	Increases storage and redundancy
Windowed Summarization	Each chunk includes a summary of previous few chunks	Long documents with sequential context	Requires LLM calls per chunk
Summarization + Metadata	Summarize each section with LLM; store prev/next section info as metadata	Complex structured documents	Expensive but highly accurate

Chunk Size Trade-offs:

Chunk Size	Embedding Focus	When to Use
1 sentence (small)	Specific meaning, precise facts	Short Q&A;, fact retrieval
Multiple paragraphs (large)	Broader themes, general context	Summarization, thematic queries
By section headers	Topic-level context	Documentation, structured reports

■ **Tip:** Chunking is iterative. Experiment with different sizes and strategies. Use ChunkViz to visualize and test your chunking approach.

Chunking Challenges:

- Text mixed with images in complex documents
- Irregular text placement (multi-column layouts)
- Color-encoded focus/emphasis that carries meaning
- Flow charts with hierarchical/ordered information
- HTML Tables are very token-heavy (3x3 table = 11 tokens plain text vs 132 tokens HTML!)

Document Extraction Approaches:

Approach	Libraries	Features/Notes
Traditional	PyMuPDF	Breaks text to raw constructs; low-level; requires hard-coded rules

Layout Model	LayoutLMv3, doctr, PyPDF, Unstructured	Deep learning for text + context extraction; understands document layout
Multi-Modal LLMs	GPT-4o, Gemini 1.5, Llava, OLMo	Intrinsically understand images; still experimental; best for complex visual docs

3.5 Embedding Models

Embedding: A numerical (vector) representation of content. Semantically similar content produces vectors that are close together in vector space.

Factors to consider when choosing an embedding model:

Data/Text Properties:

- Vocabulary size — some models handle more diverse/rare words better
- Domain/Topic — finance, medical, news etc. may need domain-specific models
- Text length — match chunk size to the model's context window

Model Capabilities:

- Multi-language support — needed for non-English documents
- Embedding dimensions/size — higher dimensions = more storage cost

Practical Considerations:

- Context window limits — many models IGNORE text beyond their limit
- Privacy and cost/licensing for proprietary API-based models
- Always benchmark multiple models and choose the best balance

■■ **Important:** CRITICAL: Always use the SAME embedding model (or models trained on similar data) for both document indexing and user query embedding. Mismatched models will produce bad results because they create different vector spaces.

3.6 Databricks Data Prep Pipelines

Pipeline Type	Embedding Managed By	Key Steps
Databricks-Managed Embeddings	Databricks auto-computes embeddings	Ingest → Process (Parse, Clean, Chunk) → Store in Delta Tables → Auto-sync to Vector Search
User-Managed Embeddings	User computes and saves embeddings manually	Ingest → Process → Embed → Store chunks+vectors in Delta Tables → Sync to Vector Search
Structured Data (Feature Store)	Feature engineering pipeline	Ingest → Feature Engineering → Delta Tables → Online Tables → Feature Serving

Module 4: LangChain Agents

Chains, Agents, ReAct & Memory

4.1 LangChain Components Overview

LangChain is a framework with 6 core components:

Component	Description
Models	LLM and chat model integrations (GPT-4, Claude, etc.)
Prompts	Prompt templates, few-shot prompts, output parsers
Chains	Sequences of steps: prompt → model → parser
Indexes	Document loaders, text splitters, vector stores for RAG
Memory	State persistence across conversation turns
Agents	Dynamic decision-making systems that choose tools at runtime

4.2 Chains vs. Agents

Aspect	Chains	Agents
Definition	A sequence of steps: input → process → output	A higher-level system that dynamically decides which tools/actions to use
Workflow	Predefined, fixed workflow	Flexible; chosen dynamically at runtime based on input
Determinism	Deterministic — always same sequence	Non-deterministic — LLM reasons to choose next step
Complexity	One main task (e.g., summarization, QA)	Complex, multi-step problem solving
Example	template | model | parser (LCEL pipeline)	Search engine + calculator + custom API calls
Use Case	Simple, predictable tasks	Tasks requiring reasoning + multiple external tools

4.3 LangChain Agents — Deep Dive

Agent: A system that uses an LLM to decide which actions to take, which tools to call, what arguments to pass, and when to stop.

- Use an LLM model to decide actions — model determines tool selection, arguments, and termination
- Run in a loop until completion: call tool → observe result → reason again → repeat
- Tools are Functions/API Calls that the agent can invoke
- Execute on a LangGraph runtime (compiled graph-based execution engine)

- Nodes = model, tools, middleware; Edges determine flow between nodes
- Agents are created via create_agent() function

Code Example: Creating an Agent

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import AzureChatOpenAI

@tool
def add(a: int, b: int) -> int:
""" adds two numbers """
return a + b

tools = [add]
model = AzureChatOpenAI(...) # Initialize the LLM
agent = create_agent(model, tools)

result = agent.invoke({
"messages": [{"role": "user", "content": "What is 12 + 7?"}]
})
print(result["messages"][-1].content) # Output: 19
```

4.4 The @tool Decorator

The **@tool** decorator converts a Python function into a LangChain Tool that agents can use.

When you add @tool, LangChain automatically:

- Registers the function as a Tool
- Uses type hints to infer an input schema (JSON schema)
- Uses the docstring as the tool description shown to the LLM
- Exposes the function so agents can call it during reasoning

@tool Usage Variants:

```
# Basic @tool (function name becomes tool name)
@tool
def add(a: int, b: int) -> int:
""" adds two numbers """
return a + b

# Custom tool name
@tool("add_two_numbers")
def method_1(a: int, b: int) -> int:
""" adds two numbers """
return a + b

# Custom name + custom description
@tool("add_two_numbers", description="add two integers")
```

```
def method_1(a: int, b: int) -> int:
""" adds two numbers """
return a + b

# Calling a tool directly
add_two_numbers.invoke({'a': 10, 'b': 5}) # Returns 15
```

■■ **Important:** The docstring is CRITICAL — the agent reads it to decide WHAT the tool does and WHETHER to call it. Always write clear, descriptive docstrings.

4.5 Tool Invocation Internals

When you define a @tool, LangChain inspects your code via Pydantic/reflection and extracts:

- **Function Name** — e.g., get_weather (used as the tool identifier)
- **Docstring** — becomes the description telling the LLM WHEN to use this tool
- **Type Hints** — define the parameters (e.g., location: str) as a JSON schema

LangChain converts this into a **JSON schema** — the format model providers (OpenAI, Anthropic, Google) expect.

create_agent() provides the "glue" — it iterates tools, converts each to JSON schema, and binds them to the LLM.

4.6 ReAct Pattern (Reason + Act)

ReAct is the core execution pattern for LangChain agents. It combines reasoning with acting in a loop.

ReAct Loop:

Step	Description
Thought	Agent reasons about what to do next (LLM generates internal reasoning)
Action	Agent selects a tool to call and specifies arguments
Observation	Agent receives the tool's output/result
Repeat	Agent reasons again based on observation, or produces Final Answer

ReAct Example — Invoice Discount Calculation:

```
Thought: I should look up the user's invoice total.
Action: query_sql
Action Input: SELECT SUM(total) FROM invoices WHERE customer_id=123;
Observation: 15430.25
Thought: Now I can compute the discount.
Action: calculator
Action Input: 15430.25 * 0.1
Observation: 1543.025
Final Answer: The 10% discount is $1,543.03.
```

The **@tool** decorator defines the "Act" capability. The agent (LLM) decides WHEN to act. The framework executes the function and returns the Observation.

4.7 Memory in LangChain Agents

By default, LLMs are **stateless** — they have NO memory of previous interactions. Each call is independent.

Memory solves this by maintaining conversation history across turns, allowing agents to refer back to earlier context.

How Memory Works:

The full conversation history (all previous messages) is passed in the messages list on each new call. The agent can then reference past context.

```
# Each invocation includes the full conversation history
result = agent.invoke({
"messages": [
{"role": "user", "content": "My name is Ali"},
{"role": "assistant", "content": "Hi Ali!"},
{"role": "user", "content": "What is my name?"} # Agent knows: Ali
]
})
```

Checkpointer — Automatic State Persistence:

A **Checkpointer** is a persistence layer that automatically saves the state of a graph after each execution step (superstep).

It is responsible for saving the thread state so the agent can remember past interactions.

Type	Class Name	Storage
In-Memory (temporary)	InMemorySaver	RAM — lost when process ends
SQLite (file)	SqliteSaver	Local SQLite database file
PostgreSQL	PostgresSaver	External PostgreSQL database
MongoDB	MongoDBSaver	External MongoDB database

What is Stored in Memory?

By default, agents store only the Messages list (conversation history). You can also define Custom Fields to store additional state such as User ID, Language, session data, etc.

Quick Reference & Exam Tips

Key Definitions, Comparisons & Memory Aids

Key Definitions to Memorize

Term	Definition
Prompt	Input or query given to an LLM to elicit a specific response
Prompt Engineering	Practice of designing and refining prompts to optimize AI responses
Zero-Shot Prompting	Asking the model to perform a task with NO examples
Few-Shot Prompting	Providing a few input-output examples to guide the model
Chain-of-Thought (CoT)	Asking the model to reason step-by-step using 'Let's think step-by-step'
Prompt Chaining	Output of one prompt becomes input of the next — breaks tasks into subtasks
RAG	Retrieval Augmented Generation — augments LLM prompts with retrieved external context
Knowledge Gap	The core problem RAG solves — LLM lacks specific domain/proprietary knowledge
Embedding	A numerical vector representation of content; similar content → nearby vectors
Vector Store	Database specialized for storing and searching vector embeddings
Chunking	Splitting documents into smaller pieces before embedding
Semantic Chunking	Splitting by sentence/paragraph/section using punctuation (., \n)
Fixed-Size Chunking	Splitting by a specific number of tokens — simple and cheap
Chunk Overlap	Consecutive chunks share some content to preserve context at boundaries
Delta Lake	Unified data management layer for reliable storage in Databricks
Unity Catalog	Governance layer: access control, lineage, auditing for Databricks
Chain (LangChain)	Fixed, deterministic sequence: prompt → model → parser
Agent (LangChain)	Dynamic system using LLM reasoning to choose tools at runtime
@tool Decorator	Converts a Python function into a LangChain tool; uses docstring as description
ReAct Pattern	Reason → Act → Observe loop for agent execution
Checkpointer	Persistence layer that saves agent state across conversation turns
create_agent()	LangChain function that builds and wires a ReAct agent with tools

Likely Exam Topics & Key Points

■ Prompt Engineering

- 4 components: Instruction, Context, Input/Question, Output Format
- Zero-shot = no examples; Few-shot = with examples
- CoT = step-by-step reasoning; Prompt Chaining = output feeds next input
- Limitations: model-dependent output; can't handle external knowledge → need RAG
- Tips: use delimiters, specify output format, prevent hallucination

■ RAG Architecture

- Pre-training vs Fine-tuning vs RAG — RAG is least complex and compute-intensive
- RAG solves the 'knowledge gap' problem
- 6 components: Index/Embed, Vector Store, Retrieval, Filtering/Reranking, Prompt Augmentation, Generation
- Benefits: up-to-date, reduces hallucinations, domain-specific, cost-effective
- Databricks: Vector Search (find context), Model Serving (generate), Unity Catalog (governance)

■ Data Prep for RAG

- 3 stages: Data Storage → Chunking → Embedding
- Fixed-size chunking = simple, semantic chunking = context-aware
- Chunk overlap prevents loss of context at boundaries
- Same embedding model MUST be used for both indexing and querying
- HTML tables are very token-heavy (132 tokens vs 11 for plain text)
- Embedding model selection: vocabulary, domain, text length, context window

■ LangChain Agents

- Chain = fixed workflow; Agent = dynamic, uses LLM reasoning
- @tool decorator: function name = tool name, docstring = description, type hints = schema
- ReAct loop: Thought → Action → Observation → (repeat) → Final Answer
- create_agent() wires model + tools into a LangGraph execution engine
- Memory = conversation history passed in messages list
- Checkpointer saves state: InMemorySaver (temp) vs DB-backed (persistent)

■ **Final Exam Strategy:** Focus on understanding the WHY behind each concept. Examiners often ask: Why RAG over fine-tuning? Why chunk overlap? Why same embedding model? Why does @tool need a docstring? Knowing the REASONS will help you answer any variation of these questions.